Worked for 16s

We'll build this in your existing project:

User request → FastAPI agent endpoint → Amazon Nova → publishing tool → FastAPI telemetry endpoint → Kafka

The agent understands plain English. Your Python code publishes the message and returns Kafka's acknowledgement.

1. Confirm Kafka is running

From the folder containing your Docker Compose file:

```bat
docker compose ps
```

Your working Kafka configuration should contain:

```yaml
ports:
  - "127.0.0.1:9092:9094"

environment:
  KAFKA_LISTENERS: INTERNAL://0.0.0.0:9092,EXTERNAL://0.0.0.0:9094
  KAFKA_ADVERTISED_LISTENERS: INTERNAL://kafka:9092,EXTERNAL://localhost:9092
  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: INTERNAL:PLAINTEXT,EXTERNAL:PLAINTEXT
  KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
  KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Since your producer already works, you do not need to recreate Kafka.

Ensure the topic exists:

```bat
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --
```

2. Open your Python project

```bat
cd /d E:\Datascience2026\kafkaws\kafkaproducerws
```

Use your existing activated `kafkaenv`.

Create these files:

File	Purpose
`.env`	Kafka, AWS, and API configuration
`requirements.txt`	Python dependencies
`src/kafkaprodapp/__init__.py`	Empty package file
`src/kafkaprodapp/schemas.py`	Validates telemetry
`src/kafkaprodapp/publish_agent.py`	LLM and publishing tool
`src/kafkaprodapp/api.py`	FastAPI endpoints and Kafka producer

Your existing `datapublisher.py` can remain.

3. Install dependencies

Create `requirements.txt`:

```
fastapi
uvicorn[standard]
confluent-kafka
pydantic>=2,<3
langchain-aws
langchain-core
python-dotenv
requests
```

Install:

```bat
python -m pip install -r requirements.txt
python -m pip check
```

4. Create `.env`

Place this file in `kafkaproducerws`, alongside `requirements.txt`:

```dotenv
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
KAFKA_TOPIC=bmw.telemetry
```

```
TELEMETRY_API_URL=http://127.0.0.1:8000

AWS_DEFAULT_REGION=us-east-1
BEDROCK_MODEL_ID=global.amazon.nova-2-lite-v1:0
```

Use the Bedrock model or inference profile that works in your AWS account.

The application uses your configured AWS credentials. If you use a named AWS profile, also add:

dotenv

```
AWS_PROFILE=your-profile-name
```

If you use temporary credentials through environment variables, all three are required: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN`.

Add these entries to `.gitignore`:

gitignore

```
.env
.venv/
kafkaenv/
__pycache__/
```

5. Create `schemas.py`

File: `src/kafkaprodapp/schemas.py`

Python Run

```python
from datetime import datetime

from pydantic import BaseModel, ConfigDict, Field


class TelemetryRequest(BaseModel):
    model_config = ConfigDict(
        extra="forbid",
        allow_inf_nan=False,
    )

    vehicle_id: str = Field(
        min_length=1,
        description="Vehicle identifier, for example BMW1001",
    )

    model: str = Field(
```

```
        min_length=1,
        description="Vehicle model, for example BMW iX",
    )

    battery_level: int = Field(
        ge=0,
        le=100,
        description="Battery percentage",
    )

    temperature: float = Field(
        description="Temperature in degrees Celsius",
    )

    speed: float = Field(
        ge=0,
        description="Speed in kilometres per hour",
    )


class TelemetryEvent(TelemetryRequest):
    timestamp: datetime


class AgentRequest(BaseModel):
    message: str = Field(
        min_length=1,
        max_length=4000,
    )
```

This schema is shared by the agent tool and the telemetry API.

6. Create the agent and its tool

File: `src/kafkaprodapp/publish_agent.py`

Python Run

```
import os
from functools import lru_cache
from pathlib import Path

import requests
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_core.tools import tool
from pydantic import ValidationError

from kafkaprodapp.schemas import TelemetryRequest
```

```python
PROJECT_ROOT = Path(__file__).resolve().parents[2]
load_dotenv(PROJECT_ROOT / ".env")

API_URL = os.getenv(
    "TELEMETRY_API_URL",
    "http://127.0.0.1:8000",
).rstrip("/")


@tool(args_schema=TelemetryRequest)
def publish_vehicle_telemetry(
    vehicle_id: str,
    model: str,
    battery_level: int,
    temperature: float,
    speed: float,
) -> dict:
    """Publish one vehicle telemetry event through the Kafka producer API."""

    payload = {
        "vehicle_id": vehicle_id,
        "model": model,
        "battery_level": battery_level,
        "temperature": temperature,
        "speed": speed,
    }

    try:
        response = requests.post(
            f"{API_URL}/telemetry",
            json=payload,
            timeout=(5, 30),
        )

    except requests.RequestException:
        # Do not automatically resend: the API may have received it.
        return {
            "status": "unconfirmed",
            "message": (
                "No API response was received. Delivery is unknown. "
                "Retrying could duplicate the event."
            ),
        }

    try:
        result = response.json()
    except ValueError:
        return {
            "status": "unconfirmed",
            "http_status": response.status_code,
            "message": "The telemetry API returned an unexpected response.",
        }
```

```
    if (
        response.status_code == 201
        and isinstance(result, dict)
        and result.get("status") == "delivered"
    ):
        return result

    return {
        "status": "not_confirmed",
        "http_status": response.status_code,
        "details": result,
    }


SYSTEM_PROMPT = """
You are a BMW telemetry publishing assistant.

Use publish_vehicle_telemetry only when the user requests
publishing or sending vehicle telemetry.

Rules:
1. Handle exactly one vehicle per request.
2. Require vehicle_id, model, battery_level, temperature and speed.
3. Never invent missing values.
4. If values are missing, ask for a complete request containing all fields.
5. Battery level must be between 0 and 100.
6. Speed must be nonnegative.
7. Temperature is in Celsius; speed is in km/h.
8. Call the tool once when all required values are available.
9. Do not claim delivery yourself; the application returns the API receipt.
"""


@lru_cache(maxsize=1)
def get_agent_model():
    model = ChatBedrockConverse(
        model_id=os.environ["BEDROCK_MODEL_ID"],
        region_name=os.getenv("AWS_DEFAULT_REGION", "us-east-1"),
        max_tokens=1024,
    )

    return model.bind_tools([publish_vehicle_telemetry])


def run_publish_agent(message: str) -> dict:
    reply = get_agent_model().invoke(
        [
            SystemMessage(content=SYSTEM_PROMPT),
            HumanMessage(content=message),
        ]
    )
```

```
    # The model may ask for missing information instead of calling a tool.
    if not reply.tool_calls:
        return {
            "status": "not_published",
            "answer": reply.content,
        }

    # Validate every proposed action before executing anything.
    if len(reply.tool_calls) != 1:
        return {
            "status": "not_published",
            "answer": "Please submit exactly one vehicle per request.",
        }

    call = reply.tool_calls[0]

    if call["name"] != publish_vehicle_telemetry.name:
        return {
            "status": "not_published",
            "answer": "Unsupported tool.",
        }

    try:
        arguments = TelemetryRequest.model_validate(call["args"])
    except ValidationError as error:
        return {
            "status": "not_published",
            "answer": f"Invalid telemetry: {error}",
        }

    # Execute once and return the actual receipt without an LLM rewrite.
```

`bind_tools()` gives the model the tool's name, description, and input schema. The model returns arguments; Python executes the tool. LangChain Bedrock documentation

7. Create the complete FastAPI application

Replace your previous `src/kafkaprodapp/api.py` with:

Python Run

```
import logging
import os
from contextlib import asynccontextmanager
from datetime import datetime, timezone
from threading import Lock

from confluent_kafka import KafkaException, Producer
from fastapi import FastAPI, HTTPException, Request
from starlette.concurrency import run_in_threadpool
```

```
# This module loads the project's .env file.
from kafkaprodapp.publish_agent import run_publish_agent
from kafkaprodapp.schemas import (
    AgentRequest,
    TelemetryEvent,
    TelemetryRequest,
)


logger = logging.getLogger(__name__)

TOPIC = os.getenv("KAFKA_TOPIC", "bmw.telemetry")


@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.producer = Producer(
        {
            "bootstrap.servers": os.getenv(
                "KAFKA_BOOTSTRAP_SERVERS",
                "localhost:9092",
            ),
            "broker.address.family": "v4",
            "client.id": "bmw-agent-producer",
            "acks": "all",
            "enable.idempotence": True,
            "delivery.timeout.ms": 10000,
        }
    )

    # Simple lab implementation: one publish-and-flush at a time.
    app.state.publish_lock = Lock()

    try:
        yield
    finally:
        remaining = await run_in_threadpool(
            app.state.producer.flush,
            12,
        )

        if remaining:
            logger.warning(
                "Shutdown with %s unconfirmed messages",
                remaining,
            )


app = FastAPI(
    title="BMW Agent Kafka Publisher",
    lifespan=lifespan,
```

```
)


@app.get("/")
def home():
    return {
        "application": "BMW Agent Kafka Publisher",
        "docs": "/docs",
    }


@app.post("/telemetry", status_code=201)
def publish_telemetry(
    payload: TelemetryRequest,
    request: Request,
):
    producer = request.app.state.producer

    event = TelemetryEvent(
        **payload.model_dump(),
        timestamp=datetime.now(timezone.utc),
    )

    delivery = {}

    def delivery_report(error, message):
        if error is not None:
            delivery["error"] = str(error)
        else:
            delivery["partition"] = message.partition()
            delivery["offset"] = message.offset()

    with request.app.state.publish_lock:
        try:
            producer.produce(
                topic=TOPIC,
                key=event.vehicle_id.encode("utf-8"),
                value=event.model_dump_json().encode("utf-8"),
                on_delivery=delivery_report,
            )

            # Executes delivery callbacks while waiting.
            producer.flush(12)

        except BufferError as error:
            raise HTTPException(
                status_code=503,
                detail="Kafka producer queue is full.",
            ) from error

        except KafkaException as error:
            raise HTTPException(
```

```
                status_code=503,
                detail=f"Kafka publishing error: {error}",
            ) from error

    if "error" in delivery:
        raise HTTPException(
            status_code=503,
            detail=f"Kafka delivery failed: {delivery['error']}",
        )

    if "offset" not in delivery:
        raise HTTPException(
            status_code=504,
            detail=(
                "Kafka acknowledgement timed out. Delivery is unknown; "
                "retrying may duplicate the event."
            ),
        )

    return {
        "status": "delivered",
        "topic": TOPIC,
        "partition": delivery["partition"],
        "offset": delivery["offset"],
        "event": event.model_dump(mode="json"),
    }


@app.post("/agent/publish")
def agent_publish(payload: AgentRequest):
    try:
        return run_publish_agent(payload.message)
    except Exception as error:
        logger.exception("Agent request failed")

        raise HTTPException(
            status_code=502,
            detail=(
                "Agent execution failed. Check the server terminal "
                "for AWS credentials, model access, or configuration errors."
            ),
```

Both endpoints use regular `def` because their operations block while waiting for external services. FastAPI runs these handlers in its thread pool. FastAPI documentation

8. Start the API

From your project root:

```
bat
```

```
python -m uvicorn kafkaprodapp.api:app --app-dir src --reload --host 127.0.0.1 --port 800
```

Open:

http://127.0.0.1:8000/docs

You should see:

Endpoint	Purpose
`GET /`	Basic application response
`POST /telemetry`	Publish structured JSON directly
`POST /agent/publish`	Interpret plain English and publish through the tool

9. Test direct publishing first

In Swagger, open **POST** `/telemetry` → **Try it out**.

```
JSON

{
  "vehicle_id": "BMW1001",
  "model": "BMW iX",
  "battery_level": 18,
  "temperature": 73,
  "speed": 80
}
```

Click **Execute**.

Check that the response contains:

```
JSON

{
  "status": "delivered",
  "topic": "bmw.telemetry"
}
```

The complete response also includes the partition, offset, and event.

This confirms the Kafka path independently of the LLM.

10. Test agent publishing

Open **POST** `/agent/publish` → **Try it out**.

JSON

```
{
  "message": "Publish telemetry for vehicle BMW1002, model BMW i4, battery level 45 perce
}
```

The application will:

1. Send your message and the tool schema to Amazon Nova.
2. Receive the tool call with extracted values.
3. Validate those values.
4. Call `POST /telemetry`.
5. Return Kafka's delivery receipt.

Example successful response:

JSON

```
{
  "status": "delivered",
  "topic": "bmw.telemetry",
  "partition": 1,
  "offset": 7,
  "event": {
    "vehicle_id": "BMW1002",
    "model": "BMW i4",
    "battery_level": 45,
    "temperature": 62.0,
    "speed": 85.0,
    "timestamp": "2026-09-11T02:30:00Z"
  }
}
```

Actual partition, offset, and timestamp will vary.

11. Test missing information

Submit:

JSON

```
{
  "message": "Publish telemetry for BMW1003."
}
```

The expected behavior is a `not_published` response asking for the missing model, battery level, temperature, and speed.

This agent is **stateless**. Submit a complete message in the next request:

JSON

```
{
  "message": "Publish telemetry for BMW1003, model BMW i7, battery 65 percent, temperatur
}
```

12. Read the messages from Kafka

In another terminal, from your Compose folder:

bat

```
docker compose exec kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kaf
```

Leave the consumer running and submit another agent request. You should see the vehicle ID key and its JSON event.

13. Troubleshoot by layer

Problem	What to check
`/telemetry` fails	Kafka container, listener mapping, topic, and `localhost:9092`
`/telemetry` works but agent returns `502`	FastAPI terminal for the actual Bedrock exception
AWS credentials missing or expired	Active AWS profile or temporary credentials
Bedrock `AccessDeniedException`	Permission to invoke the configured model/inference profile
Invalid model identifier	`BEDROCK_MODEL_ID` and AWS region
Tool cannot reach the API	`TELEMETRY_API_URL` must match the running API
`not_published`	Supply all five telemetry fields in one message

This is a local learning implementation that waits for acknowledgement on each publication. Each HTTP submission is a new event; producer idempotence does not deduplicate repeated API submissions.